

Pearl

Pearl is a small autonomous robot car that can search for a person, follow them, or hide from them. It runs entirely on a Raspberry Pi 5 with local AI models, no cloud required.

You type a description of who to look for (for example, "*person in a red jacket*") into a web dashboard. From then on the car explores on its own, checks possible sightings with an on-board vision model, and then either follows that person or moves out of sight, depending on the mode you pick. The only things a human controls are **set target**, **follow / hide**, and **recall / stop**.

Status: work in progress. The backend runs end-to-end in simulation (stub mode) and has a passing test suite, but it has **not yet been run on the real robot**. Expect small fixes during hardware bring-up.

What's in this repo

.

├─ README.md

├─ pearl_ui.html

└─ pearl_backend_v1/

├─ run.py

├─ pearl/

├─ scripts/

├─ tests/

├─ README.md

└─ DEPLOY.md

← you are here

← Pearl UI: the dashboard (runs in a browser on your laptop)

← Pearl Backend V1: the software that runs on the robot

← start here

← the Python package (vision, planner, actions, safety, HTTP bridge)

← hardware discovery + model-server launch scripts

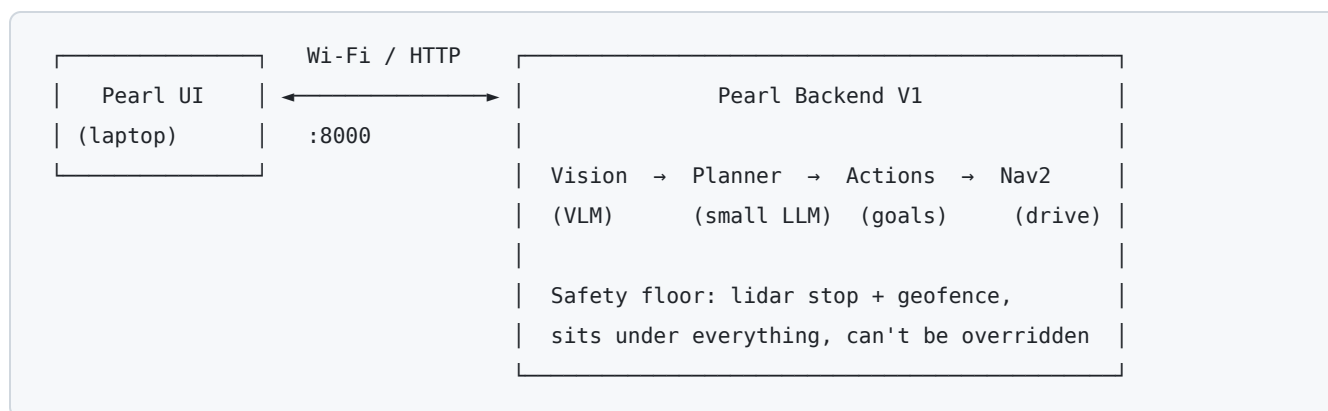
← pytest suite (runs without ROS2 or a robot)

← detailed technical docs for the backend

← step-by-step guide for putting it on the robot

Part	Runs on	What it does
Pearl UI	Your laptop's web browser	Live camera feed, current mission, action queue, system health, and the target / follow / hide / recall controls. Has a MOCK mode with fake data so you can try it with no robot.
Pearl Backend V1	The robot (Raspberry Pi 5)	Reads the camera and lidar, finds the target with a vision model, decides what to do next with a small planner model, drives with Nav2, and serves the data the dashboard displays.

How it works



1. **Vision:** a local vision-language model checks camera frames for the described person, and a lightweight tracker keeps hold of them between checks.
2. **Planner:** a small local language model looks at the situation every couple of seconds and picks the next action, such as explore a zone, approach, hold, scan with the camera, or reposition to hide.
3. **Actions:** each choice becomes a "goal" with a priority and a time limit. The action manager validates it, and newer, higher-priority goals can take over from older ones.
4. **Safety floor:** a separate layer stops the car if the lidar sees an obstacle too close, if the car nears the edge of the allowed area, or if the lidar goes silent. The AI cannot turn this off.

Both AI models run on the Pi through [llama.cpp](#).

Hardware & software

- Raspberry Pi 5-based robot car with a camera, lidar and an optional pan-tilt camera mount (built and tested against a Hiwonder MentorPi chassis)
- ROS2 Humble, with SLAM and Nav2 from the robot's stock software
- Python 3.10+ with `flask`, `requests`, `numpy`, `opencv-python-headless` and `pytest`
- llama.cpp server, plus two GGUF models: a small vision model and a small planner model

Quick start: try it with no robot

You can run everything on a laptop. The backend's **stub mode** simulates the robot, camera, lidar and AI models.

1. Start the backend in stub mode

```
cd pearl_backend_v1
pip install -r requirements.txt
python3 run.py          # stub mode is the default (PEARL_STUB=1)
```

2. Open the dashboard

Open `pearl_ui.html` in your browser. It starts in **MOCK** mode with built-in fake data.

To connect it to the stub backend you just started, set `ROBOT_BASE_URL` near the top of `pearl_ui.html` to `http://localhost:8000`, reload, and click **LIVE**.

3. Give it a target

Type something like `person in a red jacket`, click **SET TARGET**, and watch the car explore, spot the target and approach. Click **RECALL / STOP** at any time.

4. Run the tests

```
cd pearl_backend_v1
python3 -m pytest -q
```

Running on the real robot

Follow `pearl_backend_v1/DEPLOY.md` from top to bottom. It takes you phase by phase from first SSH login to the car driving itself, with a "you are done when" check at each step. In short:

```
# on the Pi
source /opt/ros/humble/setup.bash
source <hiwonder_ws>/install/setup.bash
bash scripts/discover_hw.sh | tee hw_discovery.txt  # then fill in TODO(hw) values in pearl/config.py
bash scripts/start_models.sh &                     # starts the two llama.cpp servers
PEARL_STUB=0 python3 run.py
```

Then on your laptop, set `ROBOT_BASE_URL` in `pearl_ui.html` to `http://<PI_IP>:8000` and click **LIVE**.

Configuration: what you need to fill in

This repo contains no real IP addresses, passwords or keys. Before running on a real robot, replace these placeholders **in your own local copy** and don't commit your real values back to GitHub.

Where	Placeholder	What goes there	How to find it
pearl_ui.html	ROBOT_IP:PORT in ROBOT_BASE_URL	Robot's LAN IP and backend port, e.g. http://192.168.x.x:8000	On the Pi: hostname -I (port is 8000 by default)
pearl_ui.html	CAMERA_URL	Leave it as "" to use the backend's own camera stream. Fill it in only if you use a separate stream server: http://ROBOT_IP:STREAM_PORT/stream?topic=/CAMERA_TOPIC	ros2 topic list on the Pi for the topic
Backend docs / commands	<PI_IP>	The robot's LAN IP	hostname -I on the Pi
Backend docs / commands	<hiwonder_ws>	Path to the robot vendor's ROS2 workspace (the folder containing install/setup.bash)	See Phase 1 of DEPLOY.md
Backend docs	<hiwonder-stack>	Name of the vendor's systemd service for the stock ROS2 stack	systemctl list-units --type=service grep -iE 'ros hiwonder'
pearl_backend_v1/pearl/config.py	every TODO(hw)	Topic names, chassis type, battery cells, camera field of view, and the zone / geofence / hiding-spot coordinates for your map	scripts/discover_hw.sh, plus driving around and reading /amcl_pose
pearl_backend_v1/scripts/start_models.sh	model paths	Where your GGUF model files are on the Pi	wherever you downloaded them

Optional environment variables for the backend:

Variable	Default	Purpose
PEARL_STUB	1	1 = simulated robot, 0 = real robot (ROS2 required)
PEARL_STUB_REAL_MODELS	0	1 = simulated robot, but real llama.cpp models (good for prompt tuning)
PEARL_HTTP_PORT	8000	Port the dashboard connects to
PEARL_VLM_URL	http://127.0.0.1:8080/v1/chat/completions	Vision model server
PEARL_PLANNER_URL	http://127.0.0.1:8081/v1/chat/completions	Planner model server
PEARL_LOG_LEVEL	INFO	DEBUG / INFO / WARNING
PEARL_DEBUG	1	Debug endpoints that let anyone on the network drive the robot. Set to 0 once you're past bring-up.

Dashboard ↔ robot API

The backend serves these endpoints on port `8000`, and the dashboard uses them:

Endpoint	What it does
GET <code>/api/state</code>	Mission state, battery, pose, zones, last sighting, health, safety status
GET <code>/api/goals</code>	Active goal, queue, and recent history
GET <code>/api/vision</code>	Latest detection: match, confidence, bounding box, bearing
GET <code>/camera/stream</code>	Live MJPEG camera feed
POST <code>/api/target</code>	<code>{"target": "..."} : start searching for a new target</code>
POST <code>/api/behavior</code>	<code>{"mode": "follow" "conceal"} : follow or hide on contact</code>
POST <code>/api/recall</code>	Stop everything immediately

Full payload formats are documented in the comment block at the top of `pearl_ui.html` and in `pearl_backend_v1/README.md`.

Safety notes

- **Always test with the wheels off the ground first.** `DEPLOY.md` walks through this.
- Keep the robot's **physical power switch** within reach. Recall depends on Wi-Fi and software working.
- The HTTP API has **no authentication** and is meant for a private network only. Don't expose port 8000 to the internet.
- Change the Pi's default SSH password.
- Turn off the debug endpoints (`PEARL_DEBUG=0`) for normal use.

Known limitations (v1)

- Not yet tested on real hardware.
- **Recall** stops and holds position; it doesn't drive home yet.
- **Hiding** picks the nearest preset hiding spot, without line-of-sight reasoning.
- Approach distance comes from lidar rather than the depth camera.
- The tracker is basic and meant only to bridge short gaps between vision-model checks.

The full list is in `pearl_backend_v1/README.md`.